Solution: Binary Search Tree Iterator

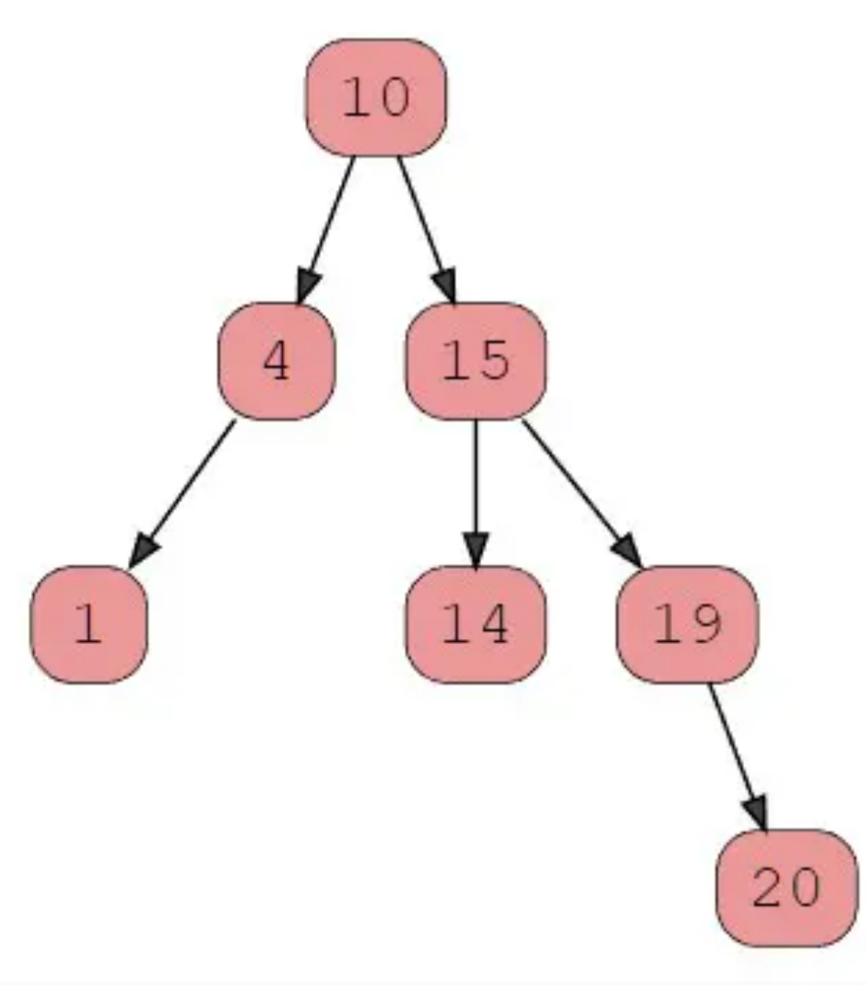
Problem Statement

Implement an iterator for the in-order traversal of a binary search tree (BST). That is, given a BST, we need to implement two functions:

bool hasNext(): Returns true if at least one element is left in the in-order traversal of the BST. **int next**(): Return the next element in the in-order traversal of the BST.

Example:

Given the following BST:



Here is the in-order traversal of this tree: [1, 4, 10, 14, 15, 19, 20]

Here is the expected output from the algorithm based on different calls:

hasNext() -> **true**

next() -> **1**

next() -> **4**

hasNext() -> **true**

next() -> **10**

next() -> **14**

next() -> **15**

next() -> **19**

next() -> **20**

hasNext() -> **false**

Solution

A brute force solution could be to store the in-order traversal of the BST in an array and used that array to process the two functions **next**() and **hasNext**(). We can maintain a pointer in the array, which points to the next element to be returned to the caller. This algorithm will take O(N) space for storing all elements in the array, and O(N) time to traverse the tree. Both the operations will take O(1) time complexity, as we have pre-processed all the data to store the in-order traversal of the BST in the array.

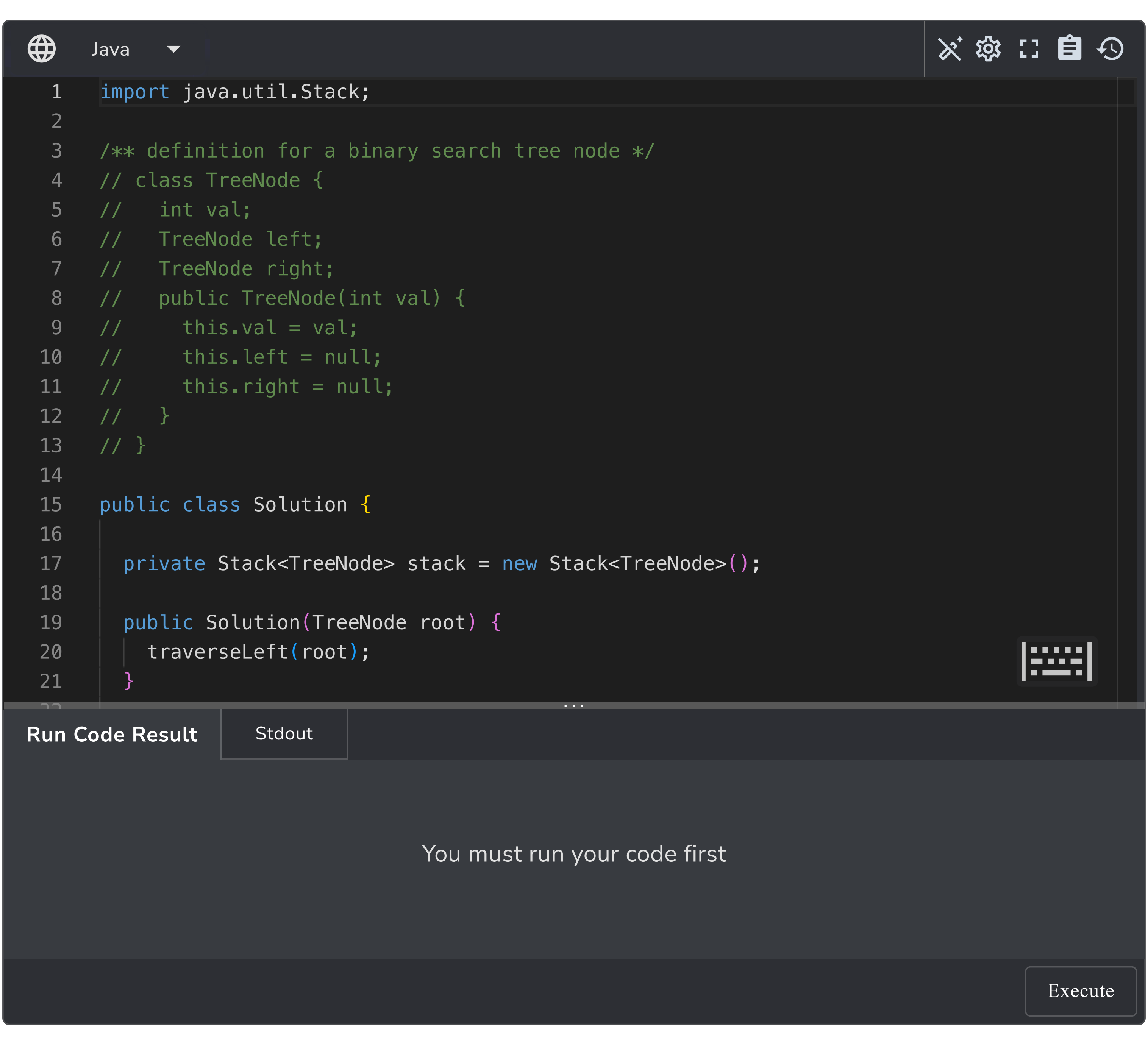
Transforming a Recursive Algorithm to an Iterative one

We know that we can perform the in-order traversal of a BST recursively. But for the iterator, we can't use recursion as we can't stop recursion in the middle to return the required element; this way, we will lose the recursive state of the traversal. Also, we didn't want to perform the in-order traversal every time **next**() or **hasNext**() is called; this will make these function O(N) operations.

We know that we can transform a recursive solution to make it iterative using a stack. This was exactly what we need! This way, we can control the recursion by saving the state of the in-order traversal of the BST in the stack, and later we can resume the tree traversal when **next**() is called again.

Code

Here is what our algorithm will look like:



```java
import java.util.Stack;

/** definition for a binary search tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {

  private Stack<TreeNode> stack = new Stack<TreeNode>();

  public Solution(TreeNode root) {
    traverseLeft(root);
  }
```

You must run your code first

Time Complexity

- The **hasNext**() will take O(1) time.
- Since **next**() calls the **traverseLeft**() function which traverses the left subtree and in the worst case the left subtree could contain all elements (practically it would be a list), hence, **next**() can take O(N) time in the worst case. One thing to note there though, **traverseLeft**() processes each element only once. This means, amortized cost of **traverseLeft**() will be O(1) for 'n' calls of next(), therefore, next() has O(1) amortized cost.

Space Complexity

We will need O(N) space for the stack in the worst case (when the BST is a list). Overall, we will need to store nodes equal to the height of the tree, hence,
O(H) space complexity, where H is the height of the given tree.

Making the Algorithm Thread Safe

Our algorithm does not have thread-safety. The algorithm would fail if multiple threads access the same **BSTIterator** object. We could get a synchronization issue in the next() function for the following two lines:

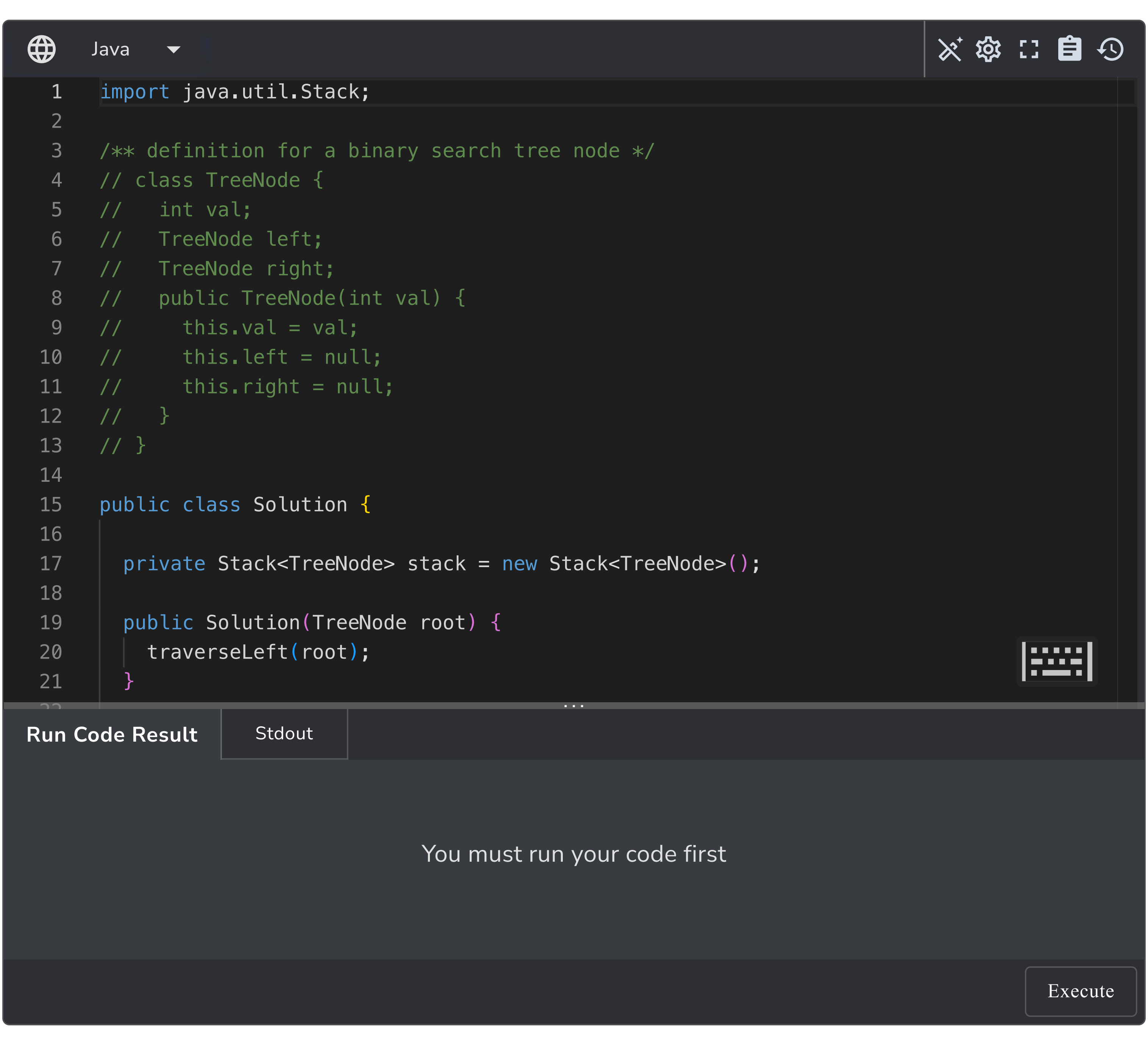
```
    TreeNode tmpNode = stack.pop();
    traverseLeft(tmpNode.right);
```

If two threads access the function concurrently, the stack could end up in a bad state.

Here is what can happen. Suppose one thread executes the first line to pop an element from the stack. Before it executes the next line (to traverse the left subtree), another thread which is also processing these lines, can also pop another element from the stack, thus, making the stack invalid.

This means we need to synchronize this function so that only one thread is allowed to process next() at any time.

Here is the new synchronized version of the algorithm.



```java
import java.util.Stack;

/** definition for a binary search tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {

  private Stack<TreeNode> stack = new Stack<TreeNode>();

  public Solution(TreeNode root) {
    traverseLeft(root);
  }
```

You must run your code first

Making the Algorithm Multi-threaded

To further improve the algorithm, we can make **next**() multi-threaded so that can return the element to the caller immediately and spawn a separate thread to perform the post-processing required to traverse the left subtree.

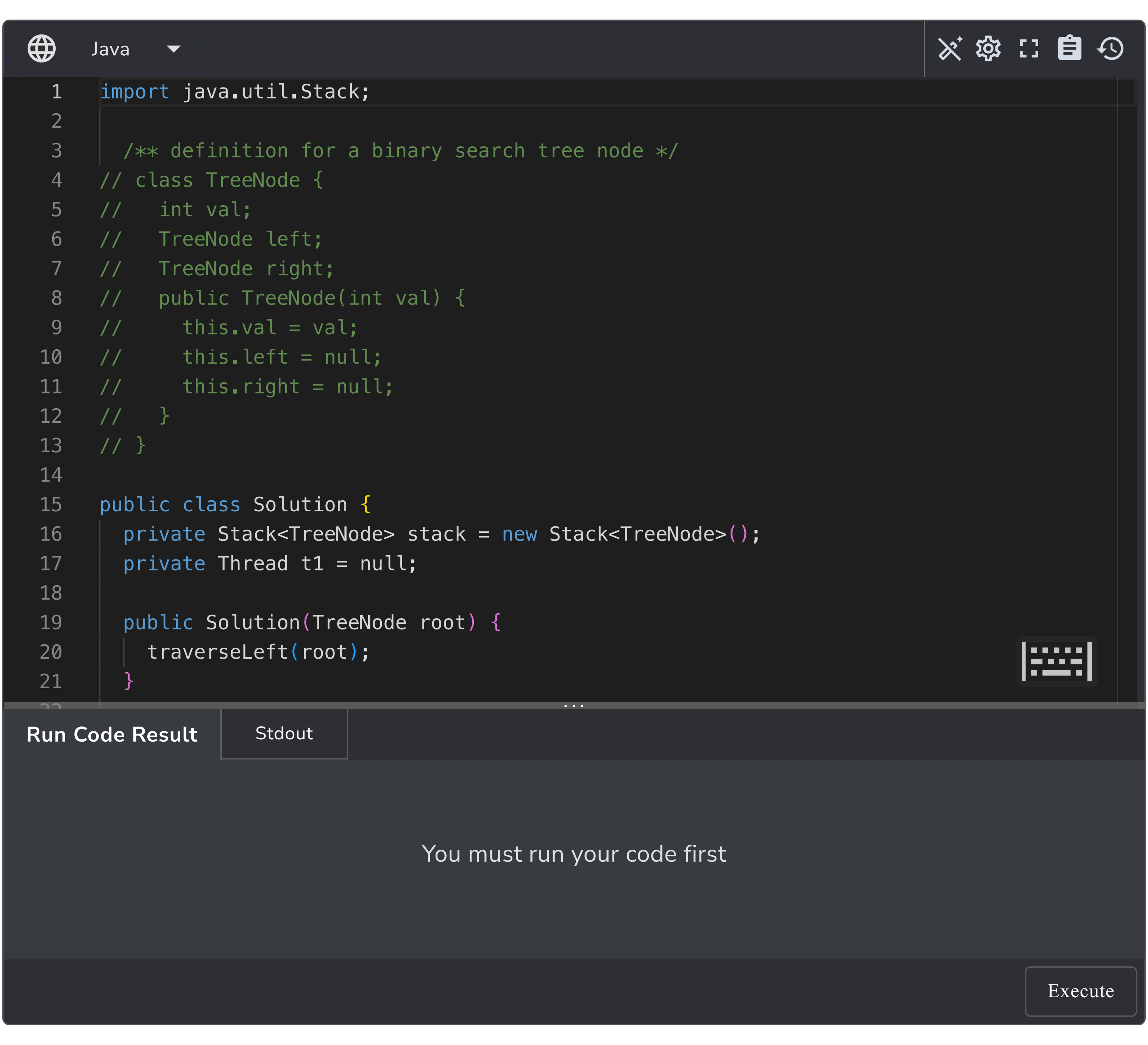
In the **next**() function, we do have the required node available right away, but we could not return to the caller before we traverse the left subtree of the right child of the current node:

`traverseLeft(tmpNode.right);`

we can spawn a new thread to process this traversal and return the element to the caller. This way, the caller is unblocked as it has the data quickly, and all the post-processing is done in a separate thread.

This made the algorithm a bit complex though. Now, whenever we are starting a new execution of **next**() or **hasNext**(), we need to ensure any previous thread doing the post-processing has finished. This means that we have to add a check before processing **next**() or **hasNext**() to wait and call **join**() on the previous thread if it has not already finished.

Here is the code that takes care of this scenario:



```java
import java.util.Stack;

  /** definition for a binary search tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {
  private Stack<TreeNode> stack = new Stack<TreeNode>();
  private Thread t1 = null;

  public Solution(TreeNode root) {
    traverseLeft(root);
  }
```

You must run your code first

Time and Space Complexities

Everything has the same complexity as the previous solution.

← Previous
Binary Search Tree Iterator (Medium)

Next →
Kth Smallest Number (Hard)

☐ Mark as Completed